

DAY 4

NAME: Nandhini V

DATE: 07-08-2025

Task:

1. Create two threads to read two separate text files
 - 1.1 Same two files, try to read using the Task Async await.
2. (Optional) Create delegate use case between teacher class and student class:
 - teacher method should have test_completed() method passed as delegate to student
 - student class should have a method write_test() which will inturn call the parent delegate.

Answers:

1. Code:

```
using System;
using System.IO;
using System.Threading;

class Program
{
    static void Main()
    {
        Thread t1 = new Thread(() => ReadFile("D:\\Test\\MyFile.txt"));
        Thread t2 = new Thread(() => ReadFile("D:\\Test\\File_3.txt"));

        t1.Start();
        t2.Start();

        t1.Join();
        t2.Join();

        Console.WriteLine("Thread-based reading complete.");
    }

    static void ReadFile(string path)
    {
        string fname = Path.GetFileName(path);
        try
        {
```

```

        string content = File.ReadAllText(path);
        Console.WriteLine($"[{Thread.CurrentThread.ManagedThreadId}] - {fname}
\nContent:\n{content}\n");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error in {fname}: {ex.Message}");
    }
}
}

```

Output:

[12] - File_3.txt
Content:
Hello

[11] - MyFile.txt
Content:
Payoda!!
Still learning

Thread-based reading complete.

2. Code:

```

using System;
using System.IO;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Task t1 = ReadFileAsync("D:\\Test\\MyFile.txt");
        Task t2 = ReadFileAsync("D:\\Test\\File_3.txt");

        await Task.WhenAll(t1, t2);

        Console.WriteLine("Async-await reading complete.");
    }

    static async Task ReadFileAsync(string path)
    {
        string fname = Path.GetFileName(path);
        try
        {

```

```

        string content = await File.ReadAllTextAsync(path);
        Console.WriteLine($"Task:- {fname} \nContent:\n{content}\n");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error in {fname}: {ex.Message}");
    }
}
}

```

Output:

Task:- File_3.txt
Content:
Hello

Task:- MyFile.txt
Content:
Payoda!!
Still learning

Async-await reading complete.

3. Code:

Student.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day4Ass3
{
    internal class Student
    {
        public delegate void pass_info(string msg);
        public pass_info pi_dele;
        public Student(pass_info pi_value)
        {
            pi_dele = pi_value;
        }
    }
}

```

```

        public void Write_Test()
        {
            pi_dele("completed the test.");
        }
    }
}

```

Teacher.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day4Ass3
{
    internal class Teacher
    {
        public Student teacher;
        public Teacher()
        {
            teacher = new Student(Test_Completed);
        }

        public void Test_Completed(string sms)
        {
            Console.WriteLine("Teacher:- The student has " + sms);
        }
    }
}

```

Program.cs

```

using Day4Ass3;

class Program
{
    static void Main(string[] args)
    {
        Teacher tr = new Teacher();
        tr.teacher.Write_Test();
    }
}

```

Output:

Teacher:- The student has completed the test.